SIMATS ENGINEERING

JAVA PROGRAMMING — ASSESSMENT 1

Student: M. Naga Shalini
Register Number: 192524183

TASK 7 — Override and Specialise Behaviour

Each vehicle category can override calculateFee(long minutes) because cars, two-wheelers and electric vehicles may have different rates. The method signature must match the superclass method; the access level cannot be more restrictive, and the return type must be compatible. Add @Override so the compiler detects spelling or signature mistakes. Use final for constants such as a fixed fee rate or for a method that must not be overridden; do not mark a method final if subclasses need different fee behaviour.

Override toString() to produce a readable vehicle description. Override equals() and hashCode() consistently if vehicles are considered equal when their normalised registration numbers match. Two equal objects must return the same hash code. A single switch on vehicle type can calculate fees in O(1) time for a fixed set of categories, just like dynamic dispatch. However, a switch must be updated when a category is added; polymorphism usually requires adding a new subclass without changing the existing fee-selection logic.

```
class Vehicle {
   String registrationNo = "TN01AB1111";
   double hourlyRate = 30.0;

   double calculateFee(long minutes) {
      long hours = Math.max(1, (minutes + 59) / 60);
      return hours * hourlyRate;
   }

   @Override
   public String toString() {
      return "Vehicle: " + registrationNo;
   }
```

```
}

class Car extends Vehicle {
    @Override
    double calculateFee(long minutes) {
        long hours = Math.max(1, (minutes + 59) / 60);
        return hours * hourlyRate;
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        System.out.println(c);
        System.out.println("Fee: " + c.calculateFee(125));
    }
}
```

TASK 8 — Apply Polymorphism

Compile-time polymorphism is method overloading: for example calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out). The compiler selects an overload using the declared argument types and number of arguments. Runtime polymorphism occurs when a Vehicle reference points to a Car or ElectricCar object and an overridden method is called. Java selects the implementation according to the actual object's class.

```
Vehicle v = new ElectricCar("TN01AB1111", 1000);
double fee = v.calculateFee(125); // ElectricCar version executes
```

The declared type is Vehicle, while the actual object is ElectricCar. The overridden ElectricCar.calculateFee method executes at runtime. An array of Vehicle references can store different subclasses, avoiding long if–else chains that test each vehicle type.

The fee computation itself is O(1) if it uses a fixed number of arithmetic operations and comparisons.

```
class Vehicle {
    double calculateFee(int minutes) {
        return minutes * 0.5;
    }

    double calculateFee(int minutes, double rate) {
        return minutes * rate;
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();
        System.out.println(v.calculateFee(60));
        System.out.println(v.calculateFee(60, 2.0));
    }
}
```

TASK 9 — Introduce Abstraction

Vehicle should be abstract because a generic vehicle may not have one universal fee rule. It can declare abstract double calculateFee(long minutes), while storing shared registration and entry details. Chargeable should be an interface because charging is a capability, not a type of vehicle; ElectricCar can implement it. A class can extend only one class but can implement multiple interfaces, allowing an ElectricCar to be both a Vehicle and Chargeable.

An abstract class can have constructors and instance fields. An interface cannot be instantiated and is useful for a contract shared by otherwise unrelated classes. Default methods provide a reusable implementation, and static interface methods provide utility behaviour associated with the interface. Changing Vehicle into an interface would make

shared instance state and constructor chaining less direct; changing Chargeable into an abstract class would consume the single class-extension slot and prevent extending another class.

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    abstract double calculateFee(long minutes);
}

class ElectricCar extends Vehicle implements Chargeable {
    double rate = 25.0;

    @Override
    double calculateFee(long minutes) {
        long hours = Math.max(1, (minutes + 59) / 60);
        return hours * rate;
    }

    @Override
    public double chargingFee(int kWh) {
        return kWh * 8.0;
    }
}

public class Main {
    public static void main(String[] args) {
        ElectricCar e = new ElectricCar();
```

```
        System.out.println("Parking fee: " + e.calculateFee(125));
        System.out.println("Charging fee: " + e.chargingFee(5));
    }
}
```

TASK 10 — Enforce Encapsulation through Packages and Access Modifiers

Organise the program into parking.model for Vehicle, Car, ElectricCar, ParkingTicket and slot-related classes; parking.service for allocation, exit and fee coordination; and parking.app for the main program. Keep fields private to protect the internal state. Use public methods for operations that other packages need, protected members only when subclasses genuinely require them, and package-private access for implementation details shared within a package.

Getters return data without exposing mutable internals. Setters or constructors must validate values, such as rejecting a blank registration number, negative duration, invalid floor or slot index. Wrapper classes such as Integer and Double allow values to be represented as objects. Avoid comparing wrapper objects with == because it checks reference identity in many cases; use equals() or compare primitive values after null checks. If fields were public, every caller would have to repeat validation, making the design inconsistent and harder to maintain.

```
class ParkingTicket {
    private String ticketId;
    private String registrationNo;

    public ParkingTicket(String id, String reg) {
        ticketId = id;
        registrationNo = reg;
    }

    public String getTicketId() {
        return ticketId;
    }
```

```
    public String getRegistrationNo() {
        return registrationNo;
    }
}

public class Main {
    public static void main(String[] args) {
        ParkingTicket p = new ParkingTicket("T101", "TN01AB1111");
        System.out.println(p.getTicketId());
        System.out.println(p.getRegistrationNo());
    }
}
```

TASK 11 — Implement, Test and Defend the Design

Testing must include normal entry and exit for each vehicle type, a full facility, the last available slot, zero-minute parking, a duration exactly on a fee boundary, an unknown vehicle type, duplicate registration numbers, an invalid slot, and exit without an active ticket. Calculate one small day's total manually and compare it with the program output. Keep the expected and actual outputs and screenshots in the repository.

Suggested test cases

1. Facility has two floors with slot counts 2 and 1. The first three unique vehicles should receive [0,0], [0,1] and [1,0]. The fourth allocation should return null. 2. Release (0,1) should return true once, false on the second attempt, and false for invalid floor 5. 3. Allocation after releasing (0,1) should reuse [0,1]. 4. Reject a duplicate active registration number before allocating another slot. 5. Reject an exit time earlier than entry time. 6. Verify the fee at one minute before, exactly at, and one minute after each fee boundary.

Complexity evaluation

For F floors and S slots per floor, a linear allocation scan is O(F×S) in the worst case and O(1) auxiliary space beyond the slot array. Release by floor and slot is O(1). Finding an active ticket in an array of n tickets is O(n); an ArrayList is also O(n) for a search by registration number. A HashMap<String, ParkingTicket> typically supports

expected O(1) lookup and update, at the cost of O(n) additional storage. An array is simple and compact for fixed capacity, while a HashMap is better for frequent ticket lookups.

Repository and evidence checklist

Create a repository named CSA09-192524183A-Assignment1 (use the exact naming rule given by your faculty). Keep one folder per task with source code and screenshots. Include README.md with student name, register number, Java version, compile/run steps and short results. Commit progressively—at least one commit per task—and add the working GitHub repository link to the report and submission form. Do not upload only one final bulk commit.

```
public class Main {
    public static void main(String[] args) {
        int slots = 3;
        int occupied = 0;

        if (occupied < slots) {
            occupied++;
            System.out.println("Vehicle parked successfully");
        } else {
            System.out.println("Parking full");
        }

        System.out.println("Occupied slots: " + occupied);
        System.out.println("Free slots: " + (slots - occupied));
    }
}
```

TASK 12 — Re-evaluate and Adapt the Design

The visible assignment describes a new requirement to add vehicle categories and a new pricing rule without rewriting classes that already work. The best response is to retain the stable allocation and release logic, but improve the fee design so each vehicle

category has its own implementation or a replaceable FeePolicy. Use a Chargeable interface for charging fees and keep charging cost separate from parking cost. Validate new categories through tests before integrating them.

Recommended design decision

Retain encapsulation, the abstract Vehicle base class and the Chargeable interface. Use polymorphic fee calculation for vehicle-specific rules and composition with a FeePolicy if rates will change frequently. Keep slot allocation independent of fee calculation. Store active tickets in a HashMap keyed by registration number if exit searches are common. For unique ticket IDs, use a shared counter for a single program run or UUIDs if multiple gates/processes issue tickets concurrently. These changes isolate future extensions and reduce the chance of breaking existing operations.

```
interface FeePolicy {
    double calculate(long minutes);
}

class StandardFeePolicy implements FeePolicy {
    double rate;

    StandardFeePolicy(double rate) {
        this.rate = rate;
    }

    public double calculate(long minutes) {
        if (minutes < 0) {
            return 0;
        }

        long hours = (minutes + 59) / 60;
        return hours * rate;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        FeePolicy f = new StandardFeePolicy(30);
        System.out.println("Parking fee: " + f.calculate(125));
    }
}
```

Limitations and validation

The simple array-based facility is suitable for a small fixed-capacity assignment, but it does not persist data after the program stops, and it does not by itself prevent concurrent allocation by multiple threads. A production system would need persistent storage, synchronisation or transactions, audit logs, payment handling and stronger ticket-ID generation. The recommendation should be supported by actual test outputs, not merely by stating that the code compiles.